

common-security

Qué hace la librería de seguridad compartida y qué hay dentro de cada uno de sus archivos: el JWT, los filtros y el enganche automático.

Este documento explica, en lenguaje simple, la librería que comparten todos los servicios de la tienda para hablar el mismo idioma de seguridad: qué es, por qué existe y qué hace cada una de sus piezas. Al final hay un recorrido paso a paso de una petición autenticada, para ver cómo encajan.

¿Qué es common-security?

No es un servicio: es una librería. Un *jar* que no se ejecuta solo, sino que `auth-service` y `negocio-service` agregan como dependencia en su `pom.xml`. Define cómo es un token JWT de GameStore, cómo se valida y cómo se convierte en una identidad que Spring Security entiende.

La idea clave es la **asimetría: `auth-service` es el único que firma tokens; todos los demás sólo los verifican.** Eso se logra con un par de claves RSA: la privada la tiene únicamente `auth-service`, la pública la tienen todos.

Lo que SÍ hace

- Firmar JWT (sólo si el servicio tiene la clave privada) y validarlos (con la pública).
- Leer las claves RSA desde archivos `.pem`.
- Convertir un token válido en un `AuthPrincipal` (id, username, rol) dentro del `SecurityContext`.
- Proteger los endpoints `/internal/**` con un secreto compartido entre servicios.
- Registrarse sola (autoconfiguración) apenas está en el classpath.

Lo que NO hace

- No tiene base de datos ni entidades: no sabe de usuarios, contraseñas ni roles reales.
- No decide qué URL es pública y cuál privada: eso lo hace el `SecurityConfig` de cada servicio.
- No emite refresh tokens ni maneja 2FA: eso es trabajo de `auth-service`.
- No la usa el `api-gateway` (es reactivo; tiene su propia versión mínima).

¿Por qué existe?

Sin esta librería, cada servicio tendría su propia copia del parser de JWT, del lector de claves y del filtro. Bastaría que una copia se desactualizara para tener un agujero. Un solo paquete, una sola forma de leer el token.

Cómo está organizada. A diferencia de `auth-service`, acá no hay subcarpetas: son siete clases en un único paquete `com.gamestore.common.security`, más un archivo de recursos. Para entenderlas conviene leerlas en tres grupos:

- **El núcleo del JWT** — `JwtProperties`, `PemKeys`, `JwtService`, `AuthPrincipal`.
- **Los filtros** — `JwtAuthenticationFilter`, `InternalTokenFilter`.
- **El enganche automático** — `CommonSecurityAutoConfiguration` + `META-INF/spring/...imports`.

El núcleo del JWT — firmar, verificar, identificar

Estas cuatro clases son el corazón de la librería: de dónde salen las claves, cómo se arma y se comprueba un token, y en qué se convierte una vez aceptado.

JwtProperties.java — los ajustes

Los valores con prefijo `app.jwt` del `application.properties` de cada servicio:

- `public-key-location` — **siempre presente**. Con esta clave se valida la firma de los tokens que llegan.
- `private-key-location` — **sólo en auth-service**. Con esta se firman los tokens nuevos. Si falta, el servicio arranca igual, pero no puede emitir.
- `access-expiration-ms` — vida del access token: 15 minutos por defecto.
- `issuer` — `gamestore-auth`. Va en el claim `iss` y todos los validadores lo exigen.

PemKeys.java — el lector de claves

Convierte los archivos `.pem` en objetos `RSAPublicKey` / `RSAPrivateKey` que jjwt puede usar. Recorta las líneas `-----BEGIN ...-----`, decodifica el Base64 y le pasa los bytes al `KeyFactory` del JDK. Sin BouncyCastle ni ninguna librería extra.

Si el archivo no existe o la clave no tiene el formato esperado, **falla al arrancar** con un mensaje que dice qué archivo leyó y qué esperaba (PKCS#8 para la privada, X.509 para la pública). Mejor un error claro al encender que uno críptico en el primer login.

JwtService.java — el que firma y verifica

La clase más importante. Se construye con la pública y, si existe, con la privada:

- `generar(uid, username, rol)` — arma el token con los claims `sub` (username), `uid` (id numérico), `rol`, `iss`, `iat` y `exp`, y lo firma con **RS256**.
- `validar(token)` — verifica la firma con la pública, exige el issuer y controla la expiración. Si algo falla lanza `JwtException`.
- `aPrincipal(claims)` — traduce los claims de un token válido a un `AuthPrincipal`.

Fail-fast a propósito

Un servicio que no debería emitir tokens no puede hacerlo por accidente. `negocio-service` se construye sin clave privada; si algún código llamara a `generar()`, explota con `IllegalStateException` en vez de firmar con una clave equivocada.

AuthPrincipal.java — la identidad

Un record con tres campos: `id`, `username` y `rol`. Es lo que queda como *principal* en el `SecurityContext` y lo que los controllers reciben con `@AuthenticationPrincipal`. Reemplaza al `CustomUserDetails` del monolito, con una diferencia grande: **no hay entidad JPA detrás**. Es sólo lo que el token afirma.

- `authorities()` — devuelve `ROLE_ADMIN`, `ROLE_VENDEDOR` o `ROLE_COMPRAADOR`, que es lo que miran las reglas `hasRole(...)`.
- `isAdmin()` — atajo para el chequeo más común.
- El `id` sirve para comparar pertenencia ("¿es tu juego?") sin ir a la base.

Los filtros — lo que corre en cada petición

Dos `OncePerRequestFilter` que cada servicio engancha en su `SecurityFilterChain`, antes del filtro de usuario/contraseña de Spring. Uno atiende al navegador, el otro a los demás servicios.

`JwtAuthenticationFilter.java` — lee el header `Authorization`

Es el equivalente servlet del filtro que tenía el monolito, pero **sin tocar la base**. En cada petición:

- 1 Busca el header `Authorization: Bearer <token>`.
- 2 Si está y todavía no hay nadie autenticado, llama a `JwtService.validar()`.
- 3 Si el token es válido, arma el `AuthPrincipal` con sus authorities y lo deja en el `SecurityContext`.
- 4 Si el token falta o es inválido, **no corta la petición**: la deja seguir anónima.

Quién dice 401 o 403

El filtro nunca rechaza nada; sólo "presenta" al usuario. Son las reglas del `SecurityConfig` de cada servicio las que deciden si esa URL necesitaba token (401) o si el rol no alcanza (403). Eso mantiene al filtro genérico y reutilizable.

`InternalTokenFilter.java` — el portón de `/internal/**`

Protege los endpoints que un servicio expone **sólo para que lo llamen otros servicios** (por ejemplo, `/internal/usuarios/by-username/{u}`, que `auth-service` consulta en cada login). Sólo actúa sobre URLs que empiezan con `/internal/`; al resto ni las mira.

- Exige el header `X-Internal-Token` con el secreto compartido (`app.internal.secret`, variable `INTERNAL_SECRET` en `docker-compose`).
- Si no coincide → 401 con `{"mensaje": "Llamada interna no autorizada"}`.
- Si coincide, marca la petición como `ROLE_INTERNAL`, que es lo que piden las reglas `.hasRole("INTERNAL")` de cada servicio.
- **No es un bean automático**: cada servicio lo instancia a mano en su `SecurityConfig`, porque cada uno decide si tiene o no API interna.

Deliberadamente simple

En un cluster real esto se reforzaría con network policies o mTLS, y el header sería una segunda barrera. Para la demo (`docker-compose` en una red interna) alcanza, y el gateway nunca expone `/internal/**` hacia afuera.

El enganche automático — cómo se instala sola

`CommonSecurityAutoConfiguration.java`

Con sólo tener la dependencia en el `pom.xml`, Spring Boot crea dos beans:

- `JwtService`, construido a partir de `JwtProperties`.
- `JwtAuthenticationFilter`, sólo si la app es servlet (por eso el gateway, que es WebFlux, no lo recibe).

Los dos llevan `@ConditionalOnMissingBean` : si un servicio quisiera reemplazarlos por una versión propia, define el suyo y la autoconfiguración se hace a un lado.

META-INF/spring/...AutoConfiguration.imports

Un archivo de una sola línea con el nombre de la clase anterior. Es el mecanismo estándar de Spring Boot 3 para decir "cuando me encuentres en el classpath, cargá esta configuración".

pom.xml — un detalle de empaquetado

Como el proyecto hereda de `spring-boot-starter-parent`, Maven intentaría convertir el jar en una aplicación ejecutable (*fat-jar*). Eso rompe una librería, así que el `repackage` está desactivado explícitamente. El `jakarta.servlet-api` es `provided` : lo pone el servicio que la use.

Las claves — la carpeta keys/ del repo

La librería lee claves, pero no las genera. Eso vive en `keys/`, en la raíz del repositorio:

- `generate-keys.sh` — genera el par con openssl (RSA de 2048 bits).
- `jwt-private.pem` — **la firma**. Sólo la lee `auth-service`.
- `jwt-public.pem` — **la validación**. La leen `auth-service`, `negocio-service` y el `gateway`.

Las claves commiteadas son de demo

Están en el repo a propósito para que `docker compose up` funcione sin pasos previos, igual que el `demo-secret` del monolito. Para un despliegue real: correr el script, **no commitear la privada**, y cargarla como `secret` o variable de entorno (`APP_JWT_PRIVATE_KEY_LOCATION`) sólo en `auth-service`.

Quién la usa y cómo

Servicio	Clave que tiene	Filtros que engancha	Nota
auth-service	privada + pública	<code>JwtAuthenticationFilter</code> + <code>InternalTokenFilter</code>	El único que llama a <code>generar()</code> . Protege <code>/api/auth/2fa/**</code> con el JWT y <code>/internal/credenciales</code> con el token interno.
negocio-service	sólo pública	<code>JwtAuthenticationFilter</code> + <code>InternalTokenFilter</code>	Valida cada request sin hablar con <code>auth-service</code> . Es 100% stateless.
api-gateway	sólo pública	— (no usa la librería)	Es WebFlux (reactivo). Tiene su propio <code>JwtValidator</code> con la misma clave pública y el mismo issuer: un pre-chequeo antes de reenviar.

Cómo encaja todo: una petición paso a paso

El recorrido completo cuando un VENDEDOR ya logueado crea un juego:

- 1 El navegador manda `POST /api/juegos` con el header `Authorization: Bearer <token>`.
- 2 El **gateway** valida el token con su `JwtValidator` (misma clave pública). Si está mal, corta con 401 y no molesta a ningún servicio.
- 3 La petición llega a **negocio-service**. `InternalTokenFilter` la ignora: no es `/internal/`.
- 4 `JwtAuthenticationFilter` lee el header y `JwtService.validar()` comprueba firma, issuer y expiración.
- 5 `aPrincipal()` arma el `AuthPrincipal(id, "vendedor1", "VENDEDOR")` y lo deja en el `SecurityContext` con `ROLE_VENDEDOR`.
- 6 Las reglas del `SecurityConfig` de `negocio-service` ven que `POST /api/juegos` pide `hasAnyRole("VENDEDOR", "ADMIN")` → pasa.
- 7 `JuegoController` recibe el `AuthPrincipal` con `@AuthenticationPrincipal` y guarda el juego con ese `username` como vendedor.

Ni un solo acceso a la base para saber quién es. Todo salió del token que `auth-service` firmó en el login.

Y una llamada entre servicios

Cuando alguien hace login, `auth-service` necesita el rol y el estado "habilitado", que viven en `negocio-service`:

- 1 `auth-service` llama a `GET /internal/usuarios/by-username/admin` agregando el header `X-Internal-Token`.
- 2 En `negocio-service`, `InternalTokenFilter` compara el secreto: coincide → `ROLE_INTERNAL`.
- 3 `JwtAuthenticationFilter` no hace nada (no hay header `Authorization`).
- 4 La regla `/internal/**` → `hasRole("INTERNAL")` deja pasar y el controller responde el perfil.

Detalles que vale la pena saber

- **RS256 en vez del HMAC del monolito.** Con HMAC, todo el que puede validar también puede firmar. Con RSA, sólo quien tiene la privada emite tokens: los demás servicios podrían verse comprometidos y aun así no podrían fabricar un token.
- **Vida corta (15 minutos).** Un JWT no se puede revocar; por eso dura poco. La sesión larga la sostiene el refresh token, que sí vive en la base de `auth-service` y sí se puede anular.
- **El claim `uid`** va en el token para que los servicios comparen pertenencia ("es tu juego", "no borres tu propio usuario") sin consultar la base.
- **Todo pasa por jjwt 0.12.6**, la misma librería en la librería compartida y en el gateway.